

Lab on Algorithmic Problems in Graph Theory

Problem 1: Stable Matching Assignment using Gale-Shapley Algorithm

In this assignment, you will implement the **Gale–Shapley stable matching algorithm** to simulate mentor–student allocation. The stable matching problem involves two sets of agents—mentors and students—each having preference lists over the other set. The goal is to produce a matching where no mentor–student pair prefers each other over their current matches.

You are required to:

- Implement the Gale–Shapley algorithm in Python using suitable data structures.
- Generate random preference lists for both mentors and students.
- Output the final stable matching and verify its stability.
- Extend the algorithm for two important variations:
 - (a) The number of mentors and students are **not the same** (e.g., 5 mentors and 7 students).
 - (b) Each mentor has a **capacity limit**—the maximum number of students they can take. Modify the algorithm to account for this constraint.
- Discuss how these variations affect the concept of stability and fairness in the resulting matching.

For background reading, refer to the [Gale–Shapley Algorithm on Wikipedia](#).

Problem 2: Two Approximation Algorithms for Vertex Cover

Given an undirected graph $G = (V, E)$, a *vertex cover* is a set $C \subseteq V$ such that every edge has at least one endpoint in C . Finding the minimum vertex cover is NP-hard. Approximation algorithms can produce near-optimal solutions efficiently.

(a) Algorithm 1: Maximal Matching-based 2-Approximation

- Compute a maximal matching M by greedily adding edges with both endpoints unmatched.
- Output $C = \{\text{both endpoints of every edge in } M\}$.

- (b) **Proof of Approximation Ratio:** Each edge in any matching requires at least one vertex in every vertex cover. Hence, $\text{OPT} \geq |M|$, where OPT is the size of the minimum vertex cover. The constructed cover has size $|C| = 2|M| \leq 2 \cdot \text{OPT}$. Thus, this algorithm is a **2-approximation**.

(c) **Algorithm 2: Greedy Degree Heuristic**

- Repeatedly pick a vertex of maximum degree, add it to the cover, and remove it and its incident edges.
- (d) Compare both algorithms on different graphs ($G(n, p)$, real-world, and dense graphs) in terms of solution size and runtime.

Complexity:

- Maximal matching: $O(m)$
- Degree-based greedy: $O((n + m) \log n)$

Problem 3: Maximal Independent Set (Practical + Theoretical Analysis)

You are given an undirected graph $G = (V, E)$. Find a *maximal* subset $S \subseteq V$ such that no two vertices in S are adjacent. You do not need to find the maximum independent set; larger $|S|$ yields a higher score.

- Problem source: [Codeforces 1376B1 – Maximal Independent Set](#)

Implement and analyze the following strategies:

- Greedy (vertex order):** Iterate through vertices in some fixed or random order; add a vertex to S if it has no neighbor already in S .
- Greedy (degree-low-first):** Repeatedly pick a vertex of minimum degree, add it to S , and remove it and all its neighbors.
- Compare results for multiple runs with random vertex orders. Report the size $|S|$ obtained for each method.

Performance Guarantee:

Let Δ be the maximum degree of G , and let $\alpha(G)$ denote the size of a maximum independent set.

- Any maximal independent set S satisfies

$$|S| \geq \frac{n}{\Delta + 1}.$$

- Also,

$$|S| \geq \frac{\alpha(G)}{\Delta + 1}.$$

Hence, any maximal independent set is a $(\Delta + 1)$ -**approximation** to the maximum independent set.

Remarks:

- No polynomial-time constant-factor approximation exists for the maximum independent set in general graphs (unless $P = NP$).
- Randomized or degree-based heuristics often yield better results in practice.